

⚡ Self-Evolving EV Data Warehouse

AI Intelligence Platform

Detailed Architecture & Code Implementation Guide

SYSTEM PHILOSOPHY

This system is a **closed-loop intelligent platform** where data flows forward through ETL and the warehouse, while intelligence flows backward through ML feedback. The key innovation is that the system **continuously evolves itself** — it rewrites its own ETL rules, discovers new dimensions, adjusts fact granularity, and optimizes its schema automatically based on ML performance.

**Data → ETL → Warehouse → Analytics → ML → Feedback → System
Evolution**

What It Does

This is the entry point of the entire system. Raw data is ingested from multiple heterogeneous sources — each with its own format, schema, and quality issues. The system is designed to handle noisy, incomplete, and inconsistent data from the start.

Datasets Used in This Project

Dataset	File	Purpose
EV Population	Electric_Vehicle_Population_Data.csv	Vehicle registrations, make/model/range
EV Charging Stations	Electric_Vehicle_Charging_Stations.csv	Station location, capacity, network
Charging Sessions	SYNTHETIC_EV_DATA.csv	Session-level energy delivery data
Global EV Sales	Global EV 2023.csv	Sales trends by country and year

Code Implementation

KAGGLE_PATHS dictionary maps logical names to actual file paths:

```
KAGGLE_PATHS = {  
    'ev_population': '/kaggle/input/datasets/utkarshx27/...',  
    'ev_stations': '/kaggle/input/datasets/sahirmaharajj/...',  
    'ev_sessions': '/kaggle/input/datasets/mexwell/...',  
    'ev_global_sales': '/kaggle/input/datasets/padmapiyush/...',  
}
```

generate_synthetic_data() is used when DRY_RUN=True, creating a minimal dataset that mirrors the real schema for fast validation without needing actual files.

Advantages

- **Abstraction:** Logical names decouple code from physical file paths — swap a dataset by changing one line.
- **Fallback safety:** DRY_RUN synthetic mode ensures the full pipeline can be tested and validated without Kaggle datasets.
- **Heterogeneous support:** Each dataset has different schemas, columns, and formats — the system handles all of them uniformly.

★ CORE INNOVATION #1: The ETL rewrites its own rules

Traditional ETL uses **fixed, hardcoded rules** (e.g., always fill nulls with mean). This system uses **ML-evaluated strategy selection** — it tries multiple methods and picks the one that actually improves model performance.

Traditional ETL

- ✗ Fixed rules (always use mean)
- ✗ Manual cleaning decisions
- ✗ Static pipeline
- ✗ Same strategy for every column

Your Intelligent ETL

- ✓ Tests mean, median, KNN, mode
- ✓ ML proxy score selects the best
- ✓ Adaptive and self-optimizing
- ✓ Logs every decision in metadata

Step-by-Step ETL Flow

Step	Method	What It Does
1. Validate	validate()	Checks null counts, data types, outlier percentages per dataset
2. Score	_proxy_score()	Trains a quick 20-tree Random Forest and measures R^2 as a quality proxy
3. Select	select_best_imputation()	Tries mean, median, KNN — picks method with highest proxy R^2
4. Transform	transform()	Applies winning strategy, logs quality score to metadata
5. Engineer	engineer_features()	Creates Vehicle Age, Range per Dollar, hour, dayofweek, is_weekend
6. Load	incremental_load()	Only loads records newer than last_loaded_time (simulated)

Code Implementation

Key method — select_best_imputation():

```
strategies = { "mean": ..., "median": ..., "knn": ... }
for method_name, fn in strategies.items():
    score = self._proxy_score(df_imputed, col) # R² from RF
    if score > best_score: best_method = method_name
```

Result logged to: METADATA_STORE['etl_runs'] — every column, every method, every score

For your real data: Since SYNTHETIC_EV_DATA.csv has columns connectionTime_decimal, chargingDuration, kWhDelivered, dayIndicator — the ETL renames kWhDelivered → kwhTotal before ML training.

Advantages

- **Self-optimizing:** No human decides how to clean data — the ML score decides.
- **Column-aware:** Each column gets its own best strategy, not a one-size-fits-all rule.
- **Auditable:** Every imputation decision is logged in METADATA_STORE with timestamp, column, method, and R^2 score.
- **Incremental:** System only processes new records since the last load, enabling near-real-time operation.

What It Does

The Staging Area is an in-memory buffer between the raw ETL output and the final warehouse. It holds cleaned data temporarily, enabling debugging, lineage tracking, and the ability to re-process data without re-running extraction.

Code Implementation

StagingArea class — key attributes:

```
self.tables = {} # staging_ev_population, staging_stations, ...
self.lineage = {} # source, rows, cols, staged_at per dataset
```

Four staging tables created:

```
staging.stage("ev_population", ev_pop_clean, "kaggle:electric-vehicle-
population")
staging.stage("stations", stations_clean, "kaggle:ev-charging-
stations")
staging.stage("sessions", sessions_clean, "kaggle:ev-charging-
sessions")
staging.stage("global_sales", global_clean, "kaggle:global-ev-dataset")
```

Advantages

- **Lineage tracking:** Every staged table records its source, row count, column list, and timestamp.
- **Isolation:** Warehouse never sees raw data — only staging-certified cleaned data passes through.
- **Debuggability:** If something breaks in the warehouse, you can inspect staging tables without re-running extraction and ETL.
- **Replayability:** Staging allows re-loading into the warehouse with different schema configurations without redoing ETL.

★ CORE INNOVATION #2: The warehouse evolves its own schema

This is the most advanced layer. Unlike traditional static data warehouses where DBAs manually define every table and column, this warehouse **discovers new dimensions automatically**, changes fact granularity based on data variance, and materializes aggregations based on query patterns.

★ Feature 1: Dynamic Dimension Discovery

Problem: Traditional warehouses have fixed dimensions (Time, Location, Vehicle). If a new important factor emerges (e.g., weather), a DBA must manually add it.

Your Solution: The system trains a Random Forest, extracts feature importances, and if any feature scores above a threshold it automatically creates a new dimension table for it.

Code — discover_dimensions():

```
rf = RandomForestRegressor(n_estimators=30)
rf.fit(X, y) # trained on sessions_clean, target=kWhDelivered
importances = dict(zip(features, rf.feature_importances_))
for feat, imp in importances.items():
    if imp > threshold: # default 0.03
        CREATE Dim_Auto_{feat} # auto schema creation!
```

Applied to your data: Runs on sessions_clean with kWhDelivered as target. Features connectionTime_decimal, chargingDuration, dayIndicator are scored — high-importance ones become auto-dimensions.

★ Feature 2: Adaptive Fact Granularity

Problem: One-size-fits-all granularity wastes storage for stable regions and loses detail for volatile ones.

Your Solution: The warehouse calculates the Coefficient of Variation (CV = std/mean) per group. High-variance groups get row-level detail; low-variance groups get aggregated summaries.

Code — build_adaptive_fact_fast():

```
stats = df.groupby(group_col)[measure_col].agg(["std", "mean"])
stats["cv"] = stats["std"] / (stats["mean"].abs() + 1e-9)
if cv > 0.3: # HIGH variance
    granularity = "detailed" # keep every row
else:
    # LOW variance
    granularity = "aggregated" # collapse to mean + count
```

Applied to your data: Fact_ChargingSessions groups by dayIndicator — days with volatile charging patterns keep every session row. Stable days are summarized.
Fact_EVPopulation groups by County.

★ Feature 3: Query-Driven Schema Evolution

Problem: Schema is designed once and never adapts to how users actually query the data.

Your Solution: The system logs every query pattern (columns accessed, frequency). When column pairs are accessed together above a threshold, it materializes a pre-aggregated view for them.

Code — evolve_schema_from_queries():

```
dw.log_query("sales_by_region", ["County", "Electric Range"], freq=5)
dw.log_query("utilization_by_day", ["dayIndicator", "kWhDelivered"], freq=4)
dw.log_query("demand_by_make", ["Make", "Electric Range"], freq=3)
# System detects County+Electric Range appear freq=5 times together
# Auto-creates: Agg_County_Electric_Range table
```

★ Feature 4: Prediction-Aware Schema

Problem: When ML model accuracy drops, there is no mechanism to diagnose whether the schema itself is the cause.

Your Solution: The feedback loop checks model scores. If R^2 drops below 0.70, the system triggers schema diagnosis — identifying low-importance features, flagging missing dimensions, and suggesting structural changes.

Code — prediction_aware_evolve():

```
if model_score < 0.70:
    low_imp = [f for f,v in feature_importances.items() if v < 0.01]
    action = "flag_low_importance_features: " + str(low_imp)
    # Logged to METADATA_STORE["feedback_actions"]
```

All Dimensions in Your Warehouse

Dimension Table	Type	Key Attributes
Dim_Time	Core	dayIndicator, connectionTime_decimal, chargingDuration, Time_key
Dim_Vehicle	Core	Make, Model, Model Year, Electric Vehicle Type, Electric Range, Base MSRP, Is_BEV, Vehicle_key
Dim_Location	Core	County, City, State, Postal Code, Location_key

Dimension Table	Type	Key Attributes
Dim_Station	Core	Station Name, Street Address, City, EV Level2 EVSE Num, EV DC Fast Count, Station_key
Dim_Auto_*	Auto-Discovered	Created dynamically by ML if feature importance > 0.03 threshold

Fact Tables in Your Warehouse

Fact Table	Granularity Control	Key Columns
Fact_ChargingSessions	By dayIndicator (CV)	kWhDelivered, dayIndicator, granularity, record_count
Fact_EVPopulation	By County (CV)	Electric Range, County, granularity, record_count
Agg_* (materialized)	Query-driven	Pre-aggregated mean, sum, count for frequent column pairs

What It Does

OLAP (Online Analytical Processing) enables multi-dimensional analysis of the warehouse data. It provides four standard operations that allow users to explore data at different levels of detail and from different angles.

Operation	Method	Example	Result
Roll-Up	roll_up()	Station → City → County	Aggregates sales upward
Drill-Down	drill_down()	County → Station level	Breaks detail downward
Slice	slice_df()	Filter to Make=Tesla	Single dimension filter
Dice	dice_df()	Tesla + King County	Multi-dimension filter

Advantages

- **Business-ready:** Analysts can explore data without writing SQL — the OLAP Engine exposes clean Python methods.
- **Multi-dimensional:** Data can be viewed from time, location, vehicle, and station perspectives simultaneously.
- **Warehouse integration:** Operates directly on fact and dimension tables, respecting the star schema structure.

What It Does

The ML layer is the intelligence engine of the platform. It trains five different models on the warehouse data, each serving a distinct business purpose. Model outputs then feed back into the warehouse to drive schema evolution.

Model	Algorithm	Target	Output Used For
Demand Forecasting	XGBoost / GBR	kwhTotal (kWh)	Predict future charging demand
Sales Trend	Random Forest	value (sales)	Forecast EV adoption trends
Station Utilization	Gradient Boosting	avg_power_kw	Optimize charging infrastructure
Customer Segmentation	K-Means + DBSCAN	Clusters	Identify EV owner profiles
Anomaly Detection	Isolation Forest	Anomaly flag	Flag abnormal sessions (fraud/faults)

Code Implementation

Sampling for performance (added for your real datasets):

```
sessions_ml = sessions_clean.sample(min(5000, len(sessions_clean)))  
ev_pop_ml   = ev_pop_clean.sample(min(5000, len(ev_pop_clean)))
```

Why this works: ML models reach near-peak accuracy on 5000 representative samples. Training on 100k+ rows would take 10x longer with marginal accuracy gain. This is standard practice in production ML.

Feature Importances extracted after each model:

```
self.feature_importances["demand_forecasting"] = dict(zip(features,  
model.feature_importances_))
```

These importances flow directly into the Feedback Loop to trigger dimension discovery and schema evolution.

Advantages

- **Multi-target:** Five different prediction targets cover demand, sales, utilization, segmentation, and anomalies — comprehensive business intelligence.
- **Feature-driven schema:** Feature importances are not just outputs — they actively reshape the warehouse structure through the feedback loop.
- **Scalable:** Sampling strategy allows training on large real-world datasets without timeouts.

★ CORE INNOVATION #3: The system improves itself automatically

This is what makes this architecture truly novel. Most data warehouses are **static** — they are built once and maintained manually. This system has a **closed feedback loop** where ML outputs directly trigger structural changes to the ETL rules, dimensions, and schema.

Full Feedback Loop Flow

ML Output (R^2 scores, feature importances)



Performance Evaluation (score > 0.5 ? ✓stable : ✗ trigger)

**Trigger Actions:**

- Change ETL imputation rules
- Auto-create new dimensions (Dim_Auto_*)
- Adjust fact granularity thresholds
- Flag low-importance features for removal



System Evolves → Retrain → Better Accuracy → Loop Again

Code Implementation**evaluate_and_trigger() — core feedback logic:**

```
for model_name, scores in ml_scores.items():
    primary_score = scores.get("R2", scores.get("silhouette", 0))
    if primary_score < 0.5: # poor performance → trigger
        dw.prediction_aware_evolve(primary_score, fi=feature_importances)
        # → creates new dims, flags low-importance features
```

History logged: Every iteration's scores, actions taken, and timestamps are stored in feedback.history and METADATA_STORE for full auditability.

Advantages

- **Self-healing:** When model accuracy drops, the system diagnoses and fixes the root cause automatically.
- **Continuous improvement:** Each feedback iteration makes the warehouse smarter — better dimensions, better granularity, better ETL rules.
- **Zero manual intervention:** No DBA needs to restructure the schema — ML drives all structural decisions.

- **Fully logged:** Every action triggered by the feedback loop is recorded in METADATA_STORE["feedback_actions"] with iteration number, score, and action taken.

What It Does

The final layer converts all warehouse data and ML outputs into actionable business insights through a dark-themed 24x20 matplotlib dashboard with 12+ panels.

Dashboard Panel	Business Question Answered
Demand Forecasting	How much kWh will be needed? (Actual vs Predicted)
Sales Trend	Which regions/powertrains are growing fastest?
Customer Segments	What types of EV owners exist? (K-Means PCA plot)
Anomaly Detection	Which charging sessions are suspicious or faulty?
Feature Importance	What drives energy demand most? (bar chart)
Schema Evolution	How has the warehouse structure changed over time?

Advantages

- **Decision-ready:** Each panel directly answers a business question about EV infrastructure, demand, or customer behavior.
- **Schema-aware:** Visualization reflects the current evolved state of the warehouse — new auto-dimensions appear in charts automatically.
- **Self-contained:** Single 24×20 figure with GridSpec layout covers all 5 ML targets plus warehouse metadata in one view.

□ Complete System Architecture Summary

Layer	Component	Class / Method	Key Innovation
1	Data Sources	load_datasets()	Multi-source ingestion with synthetic fallback
2	Intelligent ETL	IntelligentETL	ML-scored imputation strategy selection — rewrites its own rules
3	Staging Area	StagingArea	Lineage-tracked intermediate buffer with 4 staging tables
4	Adaptive Warehouse	AdaptiveDataWarehouse	Dynamic dims, adaptive granularity, query-driven materialization
5	OLAP Engine	OLAPEngine	Roll-up, drill-down, slice, dice on star schema
6	ML Pipeline	MLPipeline	Five models: forecasting, trend, utilization, segmentation, anomaly
7	Feedback Loop	FeedbackLoop	ML scores trigger automatic ETL rewrites and schema evolution
8	BI Dashboard	matplotlib / GridSpec	12-panel dark dashboard covering all business questions

Three Core Innovations That Make This System Advanced

#1 Self-Rewriting ETL

ETL tries multiple strategies, uses ML proxy R^2 to pick the best imputation rule per column, and logs every decision.

#2 Self-Evolving Schema

Warehouse discovers new dimensions from ML feature importance, adjusts fact granularity by CV, and materializes views from query patterns.

#3 Closed Feedback Loop

ML performance continuously drives structural improvements — the system gets smarter with every iteration without any manual intervention.